Go Basics Cheat Sheet

For Python and C/C++ coders starting their first Go project

WHAT IS GO

- Compiled like C; syntax reads somewhere between plain C and Python
- Garbage collected: no manual `free` / `delete`
- Built-in concurrency: goroutines and channels (no manual threads)
- No classes or inheritance; interfaces give duck-typing-like behaviour
- Generics since Go 1.18

Strengths

- Fast compiles; a single binary, no runtime to install
- Small language (25 keywords), one style enforced by `gofmt`
- Strong standard library: networking, HTTP, JSON, files

Weaknesses

- Verbose error handling (`if err != nil` everywhere)
- Less memory control than C/C++/Rust, and usually slower
- Fewer features by design: no exceptions, no inheritance
- Nil pointer crashes are still possible at runtime

RUNNING CODE

```
package main

import "fmt"

func main() {
    fmt.Println("hello")
}
```

- `go run main.go`: compile and run, binary is thrown away
- `go build -o hello`: compile and keep the binary
- `go vet ./...`: check for suspicious mistakes (`./...` = this folder and every folder below)
- `gofmt file.go` prints only; `go fmt ./...` or `gofmt -w .` rewrites files
- `golangci-lint run ./...`: stricter, bundles many linters (separate install)

PACKAGES & IMPORTS

```
import (
    js "encoding/json"   // alias, like `as js`
    "unicode/utf8"       // full path to ONE package
)
n := utf8.RuneCountInString("Intéressant!")
data, _ := js.Marshal(n)   // use the last part / alias
```

- A package = a folder of `.go` files, each starting with `package <name>`
- A program needs one `package main` with one `func main()`
- Unused imports **and** unused variables are compile errors

NAMING & VISIBILITY

```
func ParseInput() {}   // public: other packages can call it
func parseInput() {}   // private to this package
var MaxUsers = 10      // public
var userCount = 0      // private
```

- camelCase / PascalCase; snake_case only for file names
- **Uppercase** first letter = public (exported); **lowercase** = private to the package
- Applies to functions, types, struct fields and variables
- Enforced by the compiler (Python's `_private` is only a convention)
- The boundary is the package, not the struct: no class-level privacy

VARIABLES & ZERO VALUES

```
var a int            // 0
var b int = 5
g := 5               // type inferred, functions only
const maxSize = 100  // compile-time, basic types only

var s string         // ""
var ok bool          // false
var p *int           // nil
var nums []int       // nil, len 0: append still works
var ages map[string]int  // nil: reading ok, adding panics
```

- Every variable starts at its zero value, never garbage like in C

NUMBERS & BYTES

- `int` (32/64 bit, default), `int8`..`int64`, `uint8`..`uint64`, `byte`=`uint8`
- `float64` (default), `float32`, `bool`
- `7 / 2` is 3 (truncates like C); no `//` operator (that's a comment)
- Overflow wraps silently: `int8` 127 + 1 = -128 (`math.MaxInt8`, ...)
- `[]byte`: raw bytes for files and network; `[]byte(s)` / `string(b)`

ENUMS

```
type Color int
const (
    Red Color = iota   // 0, then counts up
    Green              // 1
    Blue               // 2
)
```

- No `enum` keyword: a custom int type plus a `const` block
- No value check: `Color(7)` compiles, while Python's `Color(7)` raises `ValueError`
- C++ equivalent: `enum class Color { Red, Green, Blue };`

STRINGS & RUNES

```
s := "héllo"
len(s)                      // 6 bytes, not 5
utf8.RuneCountInString(s)   // 5 runes
s = "H" + s[1:]             // immutable: new string
raw := `no \n escapes`      // like Python r"..."
msg := fmt.Sprintf("hi %s, you are %d", "Ana", 30)
m := fmt.Sprintln("hi Ana", 30)  // "hi Ana 30\n"
b := []byte(s)
b[0] = 'J'
s = string(b)               // "Jéllo"
r := 'c' + 2                // rune 101; string(r) is "e"
r = 'c' + rune(n)           // int var needs conversion
for i, ch := range s {}     // byte index, rune
```

- `"text"` is a string, `'a'` is a rune (= `int32`); no char type
- Strings are UTF-8 bytes: `len(s)` counts bytes; `"c" + 2` won't compile
- Backtick raw strings have no escapes and can span several lines
- `Sprint`/`Sprintf`/`Sprintln` return a string instead of printing (like Python f-strings)

SLICES

Python	Go
`nums = [1, 2, 3]`	`nums := []int{1, 2, 3}`
`nums.append(4)`	`nums = append(nums, 4)`
`nums[1:3]`	`nums[1:3]`
`nums[-1]`	`nums[len(nums)-1]`
`3 in nums`	`slices.Contains(nums, 3)`
`nums.sort()`	`slices.Sort(nums)`

- `append` is built in (no import); `in` isn't a keyword
- `slices` (1.21+): `Max`, `Min`, `Reverse`, `Equal`, `Insert`, `Delete`; `sort.Ints`, `sort.Strings`
- `strings`: `ToUpper`, `ToLower`, `TrimSpace`, `ReplaceAll`, `HasPrefix`, `HasSuffix`, `Split`, `Join`
- `[3]int` is a fixed-size array like C; slices are what you'll mostly use

MAPS

Python	Go
`ages = {}`	`ages := map[string]int{}`
`ages["ana"] = 25`	`ages["ana"] = 25`
`del ages["bob"]`	`delete(ages, "bob")`
`"bob" in ages`	`_, ok := ages["bob"]`

- A missing key gives the zero value, so check `ok`
- Iteration order is random (Python dicts keep insertion order)

C++98 equivalents

Go	C++98
`append(nums, 4)`	`nums.push_back(4);`
`nums[1:3]`	`std::vector<int>(nums.begin() + 1, nums.begin() + 3)`
`nums[len(nums)-1]`	`nums.back()`
`slices.Contains(nums, 3)`	`std::find(nums.begin(), nums.end(), 3) != nums.end()`
`slices.Sort(nums)`	`std::sort(nums.begin(), nums.end());`
`delete(ages, "bob")`	`ages.erase("bob");`
`_, ok := ages["bob"]`	`ages.count("bob")`

- `find`/`sort` need `<algorithm>`; `std::map` keeps keys sorted

ERRORS ARE VALUES

```
_, err := fmt.Println("hi")               // even Println returns one
err = errors.New("file is empty")         // fixed message
err = fmt.Errorf("cannot open %s", name)  // with values
```

- `error` is a type, returned like any other value; `nil` means no error
- `errors.New` needs `import "errors"`; `fmt.Errorf` formats like `Sprintf`
- Messages: lowercase, no ending punctuation

Two ways to check

```
n, err := strconv.Atoi("42")   // 1. check on the next line
if err != nil {
    return err
}
fmt.Println(n)                 // n exists here

if x, ok := ages["bob"]; !ok { // 2. check inside the if
    fmt.Println("not found")
} else {
    fmt.Println(x)
}                              // x does not exist here
```

- `ok` is a bool (map lookups, type assertions, channel receives); `err` is an `error`
- Pattern 2 limits the scope of `x` and `ok` to the if/else blocks
- More verbose than try/except, but every failure is handled where it happens

LOOPS: ONLY `FOR`

```
for i := 0; i < 5; i++ {}   // C style
for n < 100 {}              // "while"
for {}                      // forever
for i, v := range items {}  // index, value
for _, v := range items {}  // like: for v in items
for k, v := range m {}      // key, value
for k := range m {}         // key only
for i := range 10 {}        // 0 to 9 (Go 1.22+)
```

- No `while` keyword; `range` instead of `in`
- Trap: `for v := range items` gives `v` the **index**, not the value

SWITCH

```
switch n {
case 1:
    fmt.Println("one")
    fallthrough             // run the next case's body anyway
case 2, 3:                  // several values
    fmt.Println("one, two or three")
default:
    fmt.Println("other")
}                           // n == 1 prints both lines
```

- No `break` needed: the switch ends after the matching case
- `fallthrough` skips the next case's condition check
- A `case` can hold any code: an `if`, a loop, even another `switch`
- Cleaner than long if/else chains; `gofmt` lines `case` up with `switch`

POINTERS

```
x := 5
p := &x
*p = 10            // x is now 10
pt := &Point{X: 1}
pt.X = 5           // no -> needed

func newCounter() *int {
    n := 0
    return &n      // fine in Go
}
```

- Same as C: `&x`, `*p`; `nil` is **Go**'s `NULL`
- Different: no pointer arithmetic, garbage collected, returning `&local` is safe

DEFER

```
f, err := os.Open("config.txt")
if err != nil {
    return err
}
defer f.Close()    // runs when the function returns
```

- Like Python's `with`, but tied to the whole function, not a block
- Several defers run in reverse order
- C++ has no `defer`; RAII destructors are the closest match

SMALL THINGS THAT WON'T COMPILE

Error	OK
`if len(s) {`	`if len(s) > 0 {`
`n + 1.5 // n is int`	`float64(n) + 1.5`
`} (newline) else {`	`} else {`
`x = i++, ++i`	`i++ then x = i`

- No truthiness, no implicit conversions, no prefix `++`/`--`

FUNCTIONS

```
func add(a, b int) int {          // shared types grouped
    return a + b
}

func divide(a int, b int) (int, error) {
    if b == 0 {
        return 0, errors.New("cannot divide by zero")
    }
    return a / b, nil
}
result, err := divide(10, 2)
q, _ := divide(9, 3)              // _ discards a value
```

- No prototypes or headers: a function can be anywhere in the package
- `error` is always the last return value
- Failure: zero value + error. Success: result + `nil`
- Multiple returns look like tuples, but **Go** has no tuple type

STRUCTS

```
type Person struct {
    Name string
    Age  int
}
p := Person{Name: "Ana", Age: 30}
r := Person{Name: "Ana"}   // Age 0
var q Person               // Name "", Age 0

inv := Inventory{}
inv.Items = make(map[string]int)  // before adding!
```

- `p.Name` works on a struct or a pointer to one
- Other packages (incl. `encoding/json`) only see Capitalised fields; use a tag for lowercase JSON keys:
- Map fields need `make` before adding, or you get a runtime panic

```
type User struct {
    Name string `json:"name"`   // {"name":"Ann"}
    age  int                    // skipped by json.Marshal
}
```

METHODS & RECEIVERS

```
func (p Person) Greet() string {  // value: works on a copy
    return "hi, I'm " + p.Name
}
func (p *Person) Birthday() {     // pointer: changes original
    p.Age++
}
p.Birthday()                      // Go takes &p for you
```

- Value receiver: method only reads (like a C++ `const` member function)
- Pointer receiver: method changes fields, or struct is large
- Don't mix them on one type: if one needs a pointer, use pointers for all
- A method is a function with a receiver before its name; same idea as passing a struct vs a pointer in C
- Methods are what interfaces are built on (**Go**'s version of duck typing)

NEWX FUNCTIONS

```
func NewPerson(name string) *Person {
    return &Person{Name: name}
}
p := NewPerson("Ana")
```

- No constructors / `__init__`: by convention a plain `New`+type function
- Only needed when a struct needs setup; otherwise use a literal